Smart Contract Security Audit Report

[2021]

Table Of Contents

1 Executive Summary

2 Audit Methodology

3 Project Overview

3.1 Project Introduction

3.2 Vulnerability Information

4 Code Overview

4.1 Contracts Description

4.2 Visibility Description

4.3 Vulnerability Summary

5 Audit Result

6 Statement

1 Executive Summary

On 2021.10.08, the SlowMist security team received the DeCus team's security audit application for DeCus, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box lead, black, grey box assists" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.

Level	Description
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.

Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

- Reentrancy Vulnerability
- Replay Vulnerability
- Reordering Vulnerability
- Short Address Vulnerability
- Denial of Service Vulnerability
- Transaction Ordering Dependence Vulnerability
- Race Conditions Vulnerability
- Authority Control Vulnerability
- Integer Overflow and Underflow Vulnerability
- TimeStamp Dependence Vulnerability
- Uninitialized Storage Pointers Vulnerability
- Arithmetic Accuracy Deviation Vulnerability
- tx.origin Authentication Vulnerability

- "False top-up" Vulnerability
- Variable Coverage Vulnerability
- Gas Optimization Audit
- Malicious Event Log Audit
- Redundant Fallback Function Audit
- Unsafe External Call Audit
- Explicit Visibility of Functions State Variables Aduit
- Design Logic Audit
- Scoping and Declarations Audit

3 Project Overview

3.1 Project Introduction

Auidt Version:

https://github.com/decus-io/decus-core/commit/cb552eb84c7758d18c39fe2e09d0f62030830ce8

https://github.com/decus-io/decus-auction-sol/commit/ff064f3b00cacae2c90801f144afcb713e099a5c

Fixed Version:

https://github.com/decus-io/decus-core/pull/101

https://github.com/decus-io/decus-core/pull/102

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status

NO	Title	Category	Level	Status
N1	The issue of the minimum bid value	Others	Low	Confirmed
N2	Compatibility issue	Design Logic Audit	Suggestion	Confirmed
N3	Access control issue	Authority Control Vulnerability	Suggestion	Confirmed
N4	Risk of excessive authority	Authority Control Vulnerability	Medium	Confirmed
N5	Redundant code	Others	Suggestion	Confirmed
N6	Add confiscation issue	Design Logic Audit	High	Fixed
N7	Funds remaining after refund	Design Logic Audit	Low	Confirmed
N8	payExtraBurnFee issue	Design Logic Audit	High	Fixed
N9	Malleable attack risk	Others	Suggestion	Fixed

4 Code Overview

4.1 Contracts Description

The main network address of the contract is as follows:

The code was not deployed to the mainnet.

4.2 Visibility Description

The SlowMist Security team analyzed the visibility of major contracts during the audit, the result as follows:

Auction

Auction			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
bid	External	Can Modify State	inState
cancelAll	External	Can Modify State	inState
setBidResult	External	Can Modify State	inState onlyOwner
challenge	External	Can Modify State	inState
exportKeepers	External	Can Modify State	inState onlyOwner
refund	External	Can Modify State	inState
checkDeadline	Public	Can Modify State	-
getStates	Public	-	-
getUserAmount	Public	-	-
getUserBids	Public	-	-
changeState	Private	Can Modify State	-
cancelAllOrRefund	Private	Can Modify State	-
findKeeper	Private	-	-
calcCollateralAmount	Private	-	-

Airdrop			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-

Airdrop			
updateMerkleRoot	External	Can Modify State	onlyOwner
claim	External	Can Modify State	nonReentrant
refund	External	Can Modify State	nonReentrant onlyOwner
_claimRewards	Private	-	-
_stakeUnlock	Private	Can Modify State	-

BaseStaking			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
calGlobalDivident	Public	-	-
_updateRate	Internal	Can Modify State	-
_updateEndTimestamp	Internal	Can Modify State	-
_stake	Internal	Can Modify State	-
_unstake	Internal	Can Modify State	-
_claim	Internal	Can Modify State	-
_releaseReward	Internal	Can Modify State	-
_settleRewards	Internal	Can Modify State	-
_increaseStakeAmount	Internal	Can Modify State	-
_decreaseStakeAmount	Internal	Can Modify State	-
_calGlobalDivident	Internal	-	-

BaseStaking			
_updateGlobalDivident	Internal	Can Modify State	-
_divideDecimalPrecise	Internal	-	-
_multiplyDecimalPrecise	Internal	-	-

DCS			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
mint	Public	Can Modify State	-

KeeperReward			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	BaseStaking
setValidator	External	Can Modify State	onlyOwner
setTimespan	External	Can Modify State	onlyOwner
setPenalty	External	Can Modify State	onlyOwner
updateRate	External	Can Modify State	onlyOwner
updateEndTimestamp	External	Can Modify State	onlyOwner
stake	External	Can Modify State	nonReentrant
unstake	External	Can Modify State	nonReentrant
claim	External	Can Modify State	nonReentrant
accuse	External	Can Modify State	nonReentrant

KeeperReward			
appeal	External	Can Modify State	nonReentrant
winAccusation	External	Can Modify State	nonReentrant
_verifyProof	Private	-	-
_winAccusation	Internal	Can Modify State	-
_loseAccusation	Internal	Can Modify State	-
_releaseReward	Internal	Can Modify State	-

Liquidation			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
receiveFund	External	Can Modify State	-
discountPrice	External	-	-
_discountPrice	Private	-	-
calcDiscountSatsAmount	Public	-	-
assetAuction	External	Can Modify State	nonReentrant

StakingReward			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	BaseStaking
updateRate	External	Can Modify State	onlyOwner
updateEndTimestamp	External	Can Modify State	onlyOwner

StakingReward			
stake	External	Can Modify State	nonReentrant
unstake	External	Can Modify State	nonReentrant
claim	External	Can Modify State	nonReentrant

StakingUnlock			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
setLpConfig	External	Can Modify State	onlyOwner
depositLocked	External	Can Modify State	nonReentrant
stake	External	Can Modify State	nonReentrant
unstake	External	Can Modify State	nonReentrant
claim	External	Can Modify State	nonReentrant
_settleUserUnlock	Private	Can Modify State	-
_calUnlockAmount	Private	-	-
_updateMaxSpeed	Private	Can Modify State	-
_getLpSpeed	Private	-	-

SwapFeeDcs			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
getMintEthFee	Public	-	-

SwapFeeDcs			
getMintFeeAmount	External	-	-
getBurnFeeAmount	Public	-	-
payMintEthFee	External	Payable	-
payExtraMintFee	External	-	-
payExtraBurnFee	External	Can Modify State	-
collectDcs	Public	Can Modify State	onlyOwner
collectEther	Public	Can Modify State	onlyOwner

SwapFeeSats			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
getMintEthFee	Public	-	-
getMintFeeAmount	External	-	-
getBurnFeeAmount	Public	-	-
payMintEthFee	External	Payable	-
payExtraMintFee	External	-	-
payExtraBurnFee	External	-	-
collectSats	Public	Can Modify State	onlyOwner
collectEther	Public	Can Modify State	onlyOwner

SwapRewarder

SwapRewarder			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
mintReward	External	Can Modify State	onlyMinter
burnReward	External	Can Modify State	onlyMinter
abort	External	Can Modify State	onlyOwner

Vesting			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
addVesting	External	Can Modify State	onlyOwner
claim	External	Can Modify State	nonReentrant
togglePause	External	Can Modify State	onlyOwner
vestedSupply	External	-	-
lockedSupply	External	-	-
_totalVestedOf	Internal	-	-

BtcRater			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
updateRates	External	Can Modify State	onlyOwner
_updateRates	Private	Can Modify State	-

BtcRater			
calcAmountInWei	External	-	-
calcOrigAmount	External	-	-
getConversionRate	External	-	-

DeCusSystem			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
initialize	External	Can Modify State	onlyAdmin
chill	External	Can Modify State	onlyGuard
allowKeeperExit	External	Can Modify State	onlyAdmin whenNotPaused
toggleExitKeeper	External	Can Modify State	whenNotPaused
getGroup	External	-	-
addGroup	External	Can Modify State	onlyGroupAdmin whenNotPaused
deleteGroup	External	Can Modify State	whenNotPaused
deleteGroups	External	Can Modify State	whenNotPaused
getReceipt	External	-	-
getReceiptId	Public	-	-
requestMint	Public	Payable	whenNotPaused
revokeMint	Public	Can Modify State	-
forceRequestMint	Public	Payable	-

DeCusSystem			
verifyMint	Public	Can Modify State	whenNotPaused
requestBurn	Public	Can Modify State	whenNotPaused
verifyBurn	Public	Can Modify State	whenNotPaused
recoverBurn	Public	Can Modify State	onlyAdmin
getRefundData	External	-	-
refundBtc	Public	Can Modify State	onlyAdmin whenNotPaused
pause	Public	Can Modify State	onlyGuard
unpause	Public	Can Modify State	onlyGuard
_cooldown	Private	Can Modify State	-
_clearReceipt	Private	Can Modify State	-
_deleteGroup	Private	Can Modify State	-
_revokeMint	Private	Can Modify State	-
_forceRevokeMint	Private	Can Modify State	-
_verifyMintRequest	Private	Can Modify State	-
_forceVerifyBurn	Private	Can Modify State	-
_requestDeposit	Private	Can Modify State	-
_approveDeposit	Private	Can Modify State	-
_revokeDeposit	Private	Can Modify State	-
_requestWithdraw	Private	Can Modify State	-
_approveWithdraw	Private	Can Modify State	-

DeCusSystem			
_revokeWithdraw	Private	Can Modify State	-
_markFinish	Private	Can Modify State	-
_blockTimestamp	Internal	-	-
_safeAdd	Internal	-	-
_mintSATS	Private	Can Modify State	-
_burnSATS	Private	Can Modify State	-
_paySATSForBurn	Private	Can Modify State	-
_refundSATSForBurn	Private	Can Modify State	-

KeeperRegistry			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	-
updateMinKeeperCollateral	External	Can Modify State	onlyOwner
setSystem	External	Can Modify State	onlyOwner
getCollateralWei	External	-	-
isKeeperQualified	External	-	-
getKeeper	External	-	-
addAsset	External	Can Modify State	onlyOwner
updateEarlyExitFeeBps	External	Can Modify State	onlyOwner
swapAsset	External	Can Modify State	nonReentrant

KeeperRegistry			
addKeeper	External	Can Modify State	-
deleteKeeper	External	Can Modify State	nonReentrant
importKeepers	External	Can Modify State	-
punishKeeper	External	Can Modify State	onlyOwner
updateLiquidation	External	Can Modify State	onlyOwner
confiscate	External	Can Modify State	nonReentrant
addConfiscation	External	Can Modify State	-
addOverissue	External	Can Modify State	onlyOwner
offsetOverissue	External	Can Modify State	-
incrementRefCount	External	Can Modify State	onlySystem
decrementRefCount	External	Can Modify State	onlySystem
_addAsset	Private	Can Modify State	-
_blockTimestamp	Internal	-	-
_addKeeper	Private	Can Modify State	-
_refundKeeper	Private	Can Modify State	-

SATS			
Function Name	Visibility	Mutability	Modifiers
<Constructor>	Public	Can Modify State	ERC20
mint	Public	Can Modify State	-

4.3 Vulnerability Summary

[N1] [Low] The issue of the minimum bid value

Category: Others

Content

In Auction contracts, users can participate in the auction using a specific asset class through the bid function, and the number of bids must be greater than the minBidAmount parameter. However, since there is more than one asset class, and different asset classes have different values, it is unreasonable to use the same minBidAmount to check the bid quantities of different asset classes.

The same is true in the calcCollateralAmount function.

Code location: auction/contratcs/Auction.sol

```
    function bid(address _asset, uint256 _amount) external inState(State.Bid) {
        AssetMeta storage meta = assetMeta[_asset];
        require(meta.divisor != 0, "unknown asset");
        uint256 uniAmount = _amount / meta.divisor;
        require((uniAmount >= minBidAmount) && (uniAmount % minBidAmount == 0),
"invalid amount");

        UserBids storage userBids = allUserBids[msg.sender];

        if (userBids.asset == address(0)) {
            userBids.asset = _asset;
            bidderCount += 1;
        } else {
            require(userBids.asset == _asset, "bidding multiple assets is not
allowed");
        }
        userBids.uniAmount += uniAmount;

        require(ERC20(_asset).transferFrom(msg.sender, address(this), _amount));
        emit Bidded(msg.sender, _asset, _amount);
    }


    function calcCollateralAmount() private view returns (uint256) {
```

```
    uint256 collateralAmount = allUserBids[keepers[0]].uniAmount;
    for (uint256 i = 1; i < keepers.length; i++) {
        uint256 uniAmount = allUserBids[keepers[i]].uniAmount;
        if (uniAmount < collateralAmount) {
            collateralAmount = uniAmount;
        }
    }
    return collateralAmount;
}
```

Solution

It is recommended to set a minBidAmount for each asset participating in bid.

Status

Confirmed; After communicating with the project party, the project party stated that the bidding assets are all tokens anchored to BTC.

[N2] [Suggestion] Compatibility issue

Category: Design Logic Audit

Content

In the Auction contract, users can participate in the auction through the bid function. The Auction contract will transfer the user-specified amount (_amount) into the contract through the transferFrom of the asset token, and use the userBids.uniAmount parameter to record the amount of the user's bid. If the asset token is a deflationary token, it will cause the actual amount of tokens received by the contract to be inconsistent with the amount passed in by the user.

The same is true for the swapAsset function in the KeeperRegistry contract.

Code location: auction/contratcs/Auction.sol

```
function bid(address _asset, uint256 _amount) external inState(State.Bid) {
    AssetMeta storage meta = assetMeta[_asset];
    require(meta.divisor != 0, "unknown asset");
    uint256 uniAmount = _amount / meta.divisor;
    require((uniAmount >= minBidAmount) && (uniAmount % minBidAmount == 0),
```

```
"invalid amount");

        UserBids storage userBids = allUserBids[msg.sender];

        if (userBids.asset == address(0)) {
            userBids.asset = _asset;
            bidderCount += 1;
        } else {
            require(userBids.asset == _asset, "bidding multiple assets is not
allowed");
        }
        userBids.uniAmount += uniAmount;

        require(ERC20(_asset).transferFrom(msg.sender, address(this), _amount));
        emit Bidded(msg.sender, _asset, _amount);
    }
```

Solution

It is recommended to use the difference between the balance of the contract before and after the user transfer as the actual bid quantity.

Status

Confirmed

[N3] [Suggestion] Access control issue

Category: Authority Control Vulnerability

Content

In the StakingUnlock contract, the depositLocked function is used to receive and lock reward tokens from the airdrop contract, but this function can be called by any user.

Code location: core/governance/StakingUnlock.sol

```
    function depositLocked(address user, uint256 amount)
        external
        override
        nonReentrant
        returns (uint256 unlockAmount)
    {
```

```
        UserStakeRecord storage rec = userStakeRecord[user];

        unlockAmount = _settleUserUnlock(user, rec);

        userLocked[user] = userLocked[user].add(amount);
        rewardToken.safeTransferFrom(msg.sender, address(this), amount);

        if (rec.lp != IERC20(0)) _updateMaxSpeed(user, rec);

        emit DepositLocked(msg.sender, user, amount, unlockAmount);
    }
```

Solution

It is recommended to only allow the airdrop contract to call the depositLocked function.

Status

Confirmed; After communicating with the project party, the project party stated that other users will not make a profit after calling this function, so no modification will be made.

[N4] [Medium] Risk of excessive authority

Category: Authority Control Vulnerability

Content

- In the DCS contract, the DEFAULT_ADMIN_ROLE role can set any user as the MINTER_ROLE role through the grantRole function, and the MINTER_ROLE role can mint any tokens through mint.
- In the KeeperRegistry contract, the owner can modify the earlyExitFeeBps parameter through the updateEarlyExitFeeBps function, but the setting range of this parameter is not limited. If it is set incorrectly, it may cause a loss of user funds.
- In the KeeperRegistry contract, the owner can punish the keeper's stake funds through the punishKeeper function, which will lead to the risk of excessive owner authority.

Code location:

core/governance/DCS.sol

```
function mint(address to, uint256 amount) public {
    require(hasRole(MINTER_ROLE, msg.sender), "require minter role");
    _mint(to, amount);
}
```

system/KeeperRegistry.sol

```
function updateEarlyExitFeeBps(uint8 bps) external onlyOwner {
    earlyExitFeeBps = bps;
    emit EarlyExitFeeBpsUpdated(bps);
}


function punishKeeper(address[] calldata keepers) external onlyOwner {
    for (uint256 i = 0; i < keepers.length; i++) {
        address keeper = keepers[i];
        KeeperData storage data = keeperData[keeper];

        address asset = data.asset;
        uint256 amount = data.amount;
        confiscations[asset] = confiscations[asset].add(amount);
        data.amount = 0;
        emit KeeperPunished(keeper, asset, amount);
    }
}
```

Solution

- It is recommended to add a minting limit for DCS tokens, or hand over the role of DEFAULT_ADMIN_ROLE to community governance.
- It is recommended to limit the range of the earlyExitFeeBps parameter setting.
- It is recommended to hand over the punishKeeper function to community management.

Status

Confirmed; After communicating with the project party, the project party reported that the authority will be transferred

to the timelock contract in the future. At present, the contract has not been deployed on the mainnet, and the ownership of the contract has not been transferred, so there is a risk of excessive authority.

[N5] [Suggestion] Redundant code

Category: Others

Content

- In the SwapRewarder contract, the mintRewardAmount parameter and burnRewardAmount are fixed parameters set when the contract is initialized, so the second receiving parameter in the mintReward function and burnReward function is redundant.
- In the SwapFeeSats contract, the first receiving parameter defined by the payExtraMintFee function and the payExtraBurnFee function is not used.
- In the SwapFeeDcs contract, the receiving parameters defined by the getMintFeeAmount, getBurnFeeAmount, payExtraMintFee and payExtraBurnFee functions are also not used.

Code location:

core/governance/SwapRewarder.sol

```
function mintReward(address to, uint256) external override onlyMinter {
    if (mintRewardAmount == 0) return;

    dcs.safeTransfer(to, mintRewardAmount);

    emit SwapRewarded(to, mintRewardAmount, true);
}

function burnReward(address to, uint256) external override onlyMinter {
    if (burnRewardAmount == 0) return;

    dcs.safeTransfer(to, burnRewardAmount);

    emit SwapRewarded(to, burnRewardAmount, false);
}
```

core/governance/SwapFeeSats.sol

```
    function payExtraMintFee(address, uint256 amount) external view override returns
(uint256) {
        return amount.mul(mintFeeBps).div(10000);
    }

    function payExtraBurnFee(address, uint256 amount) external view override returns
(uint256) {
        return amount.mul(burnFeeBps).div(10000);
    }
```

core/governance/SwapFeeDcs.sol

```
    function getMintFeeAmount(uint256) external pure override returns (uint256) {
        return 0;
    }

    function getBurnFeeAmount(uint256) public pure override returns (uint256) {
        return 0;
    }

    function payExtraMintFee(address, uint256) external pure override returns
(uint256) {
        return 0;
    }

    function payExtraBurnFee(address from, uint256) external override returns
(uint256) {
        dcs.safeTransferFrom(from, address(this), burnFeeDcs);
        return 0;
    }
```

Solution

It is recommended to remove redundant code.

Status

Confirmed; After communicating with the project party, the project party stated that these interface parameters are reserved for subsequent contract upgrades.

[N6] [High] Add confiscation issue

Category: Design Logic Audit

Content

In the KeeperRegistry contract, any user can add funds to confiscations through the addConfiscation function. If the user authorizes the KeeperRegistry contract as uint(-1) when adding the keeper, the malicious user can specify the sender as these over-authorized users through the addConfiscation function, and use other user funds to add confiscations.

Code location: system/KeeperRegistry.sol

```
function addConfiscation(
    address sender,
    address asset,
    uint256 amount
) external override {
    IERC20(asset).safeTransferFrom(sender, address(this), amount);
    uint256 normalizedAmount = btcRater.calcAmountInWei(asset, amount);
    confiscations[asset] = confiscations[asset].add(normalizedAmount);
    emit ConfiscationAdded(asset, normalizedAmount);
}
```

Solution

It is recommended to limit the call permission of addConfiscation and check whether the passed asset parameter is in the whitelist.

Status

Fixed

[N7] [Low] Funds remaining after refund

Category: Design Logic Audit

Content

In the KeeperRegistry contract, if the keeper exits early, then the earlyExitFeeBps parameter, part of the keeper

staked funds will be left in the KeeperRegistry contract, but this contract does not reserve the corresponding interface to withdraw this part of the funds.

Code location: system/KeeperRegistry.sol

```
    function _refundKeeper(KeeperData storage data, uint256 cAmount)
        private
        returns (uint256 amount)
    {
        data.amount = data.amount.sub(cAmount);
        amount = cAmount;
        address asset = data.asset;
        if (_blockTimestamp() < data.joinTimestamp + MIN_KEEPER_PERIOD) {
            amount = amount.mul(10000 - earlyExitFeeBps).div(10000);
        }
        IERC20(asset).safeTransfer(msg.sender, btcRater.calcOrigAmount(asset,
amount));
    }
```

Solution

It is recommended to send the fee collected by earlyExitFeeBps to the specified address during _refundKeeper operation.

Status

Confirmed; After communicating with the project party, the project indicated that we planed to use liquidation contract to move funds out from KeeperRegistry. The admin will be a timelocked DAO account, so the process will be governed by the community.

[N8] [High] payExtraBurnFee issue

Category: Design Logic Audit

Content

When DeCusSystem performs the requestBurn operation, it calls the payExtraBurnFee function of the SwapFeeDcs contract to charge the user a certain burn fee, which is paid to the SwapFeeDcs contract in the form of DCS tokens,

but the payExtraBurnFee function does not have any authority Any user can call this function to maliciously transfer DCS tokens from other over-authorized users to the SwapFeeDcs contract.

Code location: system/DeCusSystem.sol

```
    function payExtraBurnFee(address from, uint256) external override returns
(uint256) {
        dcs.safeTransferFrom(from, address(this), burnFeeDcs);
        return 0;
    }

    function _paySATSForBurn(
        address from,
        address to,
        uint256 amountInSatoshi
    ) private {
        uint256 amount = (amountInSatoshi).mul(BtcUtility.getSatsAmountMultiplier());

        uint256 feeAmount = fee.payExtraBurnFee(from, amount);

        sats.safeTransferFrom(from, to, amount.add(feeAmount));

        if (feeAmount > 0) {
            sats.safeTransfer(address(fee), feeAmount);
        }
    }
```

Solution

It is recommended to restrict the payExtraBurnFee function to only be called by the DeCusSystem contract.

Status

Fixed

[N9] [Suggestion] Malleable attack risk

Category: Others

Content

The _verifyMintRequest function in the DeCusSystem contract restores the address of the signer through the

ecrecover interface, but does not check the incoming s value. Since the EVM opcode allows malleable signatures, there is a risk of malleable attacks.

Code location: system/DeCusSystem.sol

```
    function _verifyMintRequest(
        Group storage group,
        MintRequest calldata request,
        address[] calldata keepers,
        bytes32[] calldata r,
        bytes32[] calldata s,
        uint256 packedV
    ) private {
        require(keepers.length >= group.required, "not enough keepers");

        uint32 cooldownTime = _blockTimestamp() + KEEPER_COOLDOWN;
        bytes32 digest = _hashTypedDataV4(
            keccak256(abi.encode(REQUEST_TYPEHASH, request.receiptId, request.txId,
request.height))
        );

        for (uint256 i = 0; i < keepers.length; i++) {
            address keeper = keepers[i];

            require(cooldownUntil[keeper] <= _blockTimestamp(), "keeper is in
cooldown");
            require(group.keeperSet.contains(keeper), "keeper is not in group");
            require(ecrecover(digest, uint8(packedV), r[i], s[i]) == keeper, "invalid
signature");
            // assert keepers.length <= 32
            packedV >>= 8;

            _cooldown(keeper, cooldownTime);
        }
    }
```

Solution

It is recommended to use OpenZeppelin's ECDSA library.

Status

Fixed

5 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002110210001	SlowMist Security Team	2021.10.08 - 2021.10.21	Medium Risk

Summary conclusion: The SlowMist security team use a manual and SlowMist team's analysis tool to audit the project, during the audit work we found 2 high risks, 1 medium risk, 2 low risks, 4 suggestion vulnerabilities. And 1 medium risk, 2 low risks, 3 suggestion vulnerabilities were confirmed and being fixed; All other findings were fixed. The code was not deployed to the mainnet. At present, the ownership of part of the contract has not been transferred to the timelock contract and DAO, so the contract still has the risk of excessive authority.

6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.

Official Website

www.slowmist.com

E-mail

team@slowmist.com

Twitter

@SlowMist_Team

Github

https://github.com/slowmist